CLOUD-01: Cloud Integration Fundamentals

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 1 of 8 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook introduces how Dynatrace integrates with major cloud providers (AWS, Azure, GCP) to deliver full-stack observability across cloud environments. You will learn the difference between ActiveGate-based polling and API-based ingestion, how the Dynatrace entity model maps cloud resources, and how to plan a multi-cloud monitoring strategy.

Table of Contents

1. Integration Architecture
2. Connection Methods: Clouds App vs Classic
3. Cloud Entity Model
4. Cloud Metrics vs Host Metrics
5. Querying Cloud Entities
6. Multi-Cloud Strategy
7. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail (the DQL cells do not run on Dynatrace Managed)
Permissions	`storage:metrics:read`, `storage:entities:read` + `storage:smartscape:read`, `settings:objects:read`, `storage:buckets:read` (Grail IAM permissions)
Cloud Integration	At least one cloud provider connected (AWS, Azure, or GCP)
Prior Knowledge	Basic Dynatrace navigation and DQL fundamentals

1. Integration Architecture

Dynatrace supports multiple approaches for cloud integration, with **direct connections via the Clouds app** being the recommended modern approach:

Approach	Mechanism	Latency	Use Case
Clouds App (Direct Connection)	Dynatrace SaaS connects directly to cloud provider APIs — no ActiveGate needed	5 min	Recommended for SaaS: full entity discovery, topology, metrics
Classic ActiveGate Polling	Environment ActiveGate queries cloud provider APIs on a schedule	5-15 min	Legacy approach; required for Managed deployments
API-Based / Streaming	Cloud provider pushes data to Dynatrace via API (e.g., Firehose, Event Hub, Pub/Sub)	Near real-time	High-frequency metrics, log forwarding
OneAgent on Host	Agent installed directly on VMs/containers	Seconds	Deep code-level visibility, distributed tracing

How the Clouds App Works (Recommended)

1. Open the **Clouds app** in your Dynatrace environment.
2. Select the cloud provider (AWS, Azure, or GCP) and follow the guided onboarding flow.
3. For **AWS**: Dynatrace deploys a CloudFormation stack in your account that creates IAM roles, Secrets Manager entries, and optional log forwarding resources.
4. For **Azure**: Create an Azure connection in the Clouds app (Entra ID app registration with a federated identity credential and the Monitoring Reader role), or use the Azure Native Dynatrace Service from the Azure Marketplace.
5. For **GCP**: Create a GCP connection in the Clouds app (generally available — verify it has reached your tenant; keyless service account impersonation), or deploy `dynatrace-gcp-monitor` on GKE (maintenance mode; keeps working until you move), which polls Cloud Monitoring for metrics and pulls logs from Pub/Sub.
6. Dynatrace connects directly to cloud APIs — **no ActiveGate required** for SaaS deployments.
7. Discovered resources are mapped to the Dynatrace entity model, enriched with cloud-native metadata (tags, account IDs, regions).

> **Note:** Classic ActiveGate-based polling is still supported for Managed deployments and environments with strict network requirements, but direct connections are the recommended approach for SaaS.

Key Consideration: Integration vs Agent

Cloud integration (whether direct or via ActiveGate) provides **infrastructure-level** visibility (CPU, memory, network, cloud-specific metrics). For **application-level** visibility (distributed traces, code-level diagnostics), you add OneAgent, OpenTelemetry, or both.

2. Connection Methods: Clouds App vs Classic

Clouds App — Direct Connections (Recommended)

The **Clouds app** provides a streamlined, fully managed experience for connecting cloud providers to Dynatrace SaaS. No customer-deployed ActiveGate is required.

Advantages:
- No ActiveGate infrastructure to deploy or maintain
- Guided onboarding with Infrastructure-as-Code (CloudFormation for AWS)
- Unified management interface for all cloud connections
- Automatic telemetry enrichment with cloud-native metadata (tags, account IDs)
- Supports metrics, logs, topology, and events from a single connection

Supported Providers:
Provider	Status	Onboarding Method
AWS	Available	CloudFormation template via Clouds app
Azure	Available	Azure Native Dynatrace Service or Entra ID app
GCP	Available (GA 10/29/2026)	GCP connection (service account impersonation); `dynatrace-gcp-monitor` on GKE keeps working until you move

Classic ActiveGate-Based Polling (Legacy)

For Managed deployments or environments with specific network constraints, the classic ActiveGate approach is still available. On Managed, classic polling runs on Environment ActiveGates; their placement, redundancy and updates are covered in DTM-04: ActiveGate Deployment on Managed.

Advantages:
- Works with Dynatrace Managed (on-premises)
- Supports environments with outbound-only connectivity
- Single integration point behind firewalls

Limitations:
- Requires deploying and maintaining ActiveGate compute resources
- Polling interval introduces 5-15 minute data latency
- API rate limits can slow discovery in large environments
- Higher operational overhead

When to Use Which

Scenario	Recommended Approach
Dynatrace SaaS (new setup)	**Clouds app** direct connection
Dynatrace Managed	Classic ActiveGate polling

Scenario	Recommended Approach
High-frequency alerting (< 5 min)	API-based streaming (Firehose, Event Hub, Pub/Sub)
Serverless workloads (Lambda, Functions)	Clouds app + Lambda Layer for tracing
Strict network isolation	Classic ActiveGate (outbound-only)
Large-scale environments (1000+ resources)	Clouds app + selective streaming for critical metrics

> **Sources:** AWS integration (DT docs) — CloudFormation onboarding and the connection model, Azure Native Dynatrace Service (DT docs) — the Azure onboarding path, Google Cloud integration (DT docs) — the Helm/Pub-Sub deployment. **Derived:** the Clouds-app-vs-classic advantages list is this entry's comparison; no page frames them as a trade-off table. *(The AWS row previously read "Generally Available" — no source applies that lifecycle term, so it now reads "Available", matching CLOUD-05 § 4 and DBMON-01 § 7.)*

Two Directions of Authentication

A cloud connection is *Dynatrace reading your cloud*: AWS role assumption, an Azure service principal, or GCP service-account impersonation. **Workload identity federation** is the opposite direction — *"Workload identity federation (WIF) lets an external workload call the Dynatrace API with a token issued by its own identity provider."* It suits CI jobs, Azure and GCP workloads and Kubernetes pods that call Dynatrace APIs, and it is configured through the Account Management API. It is Preview: *"Workload identity federation is an Early Access feature."*

It does not replace the tokens that push paths such as Firehose and CloudWatch Metric Streams authenticate with; those send a static key. The page does not list which ingest endpoints accept a federated token, so verify before designing around it.

> **Sources:** Workload identity federation (DT docs) — *"Workload identity federation (WIF) lets an external workload call the Dynatrace API with a token issued by its own identity provider."*

Upgrading from Classic to New Cloud Connections

An environment that already runs classic connections — ActiveGate polling for AWS and Azure, the self-hosted `dynatrace-gcp-monitor` workload for GCP — does not move to Clouds-app connections by itself: *"Existing connections are not automatically upgraded or removed."* The new connection is created alongside the classic one, and everything built on classic data is re-pointed before the classic connection is switched off.

Two things change for every dashboard, alert, SLO, workflow and notebook that reads classic cloud data:

What changes	Classic connection	New cloud connection
Entities	Classic types such as `EC2_INSTANCE` and `AZURE_VM`, or `CUSTOM_DEVICE` (every GCP resource, with a `cloud:gcp:*` sub-type)	Smartscape nodes such as `AWS_EC2_INSTANCE`, `AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES`, `GCP_COMPUTE_GOOGLEAPIS_COM_INSTANCE` — classic IDs do not carry over
AWS metric keys	`dt.cloud.aws.*`, `cloud.aws.*` (Metric Streams), `builtin:cloud.aws.*`	`cloud.aws.<Service>.<Metric>.By.<Dimension>`, dimension `dt.da.source == "aws-metric-poller"`
Azure metric keys	`builtin:cloud.azure.*`, `ext:cloud.azure.*`	`cloud.azure.<namespace>.<Metric>`, dimension `dt.da.source == "azure-metric-poller"`
GCP metric keys	`ext:cloud.gcp.*`	`cloud.gcp.<service>.<Metric>`, dimension `dt.da.source == "gcp-cloud-monitoring"`

The guide's sequence, per account, subscription or project:

1. **Create the new connection** next to the classic one — start with a non-production account.
2. **Verify topology and telemetry** — inventory takes up to 15 minutes to reach Smartscape.
3. **Upgrade dependent assets** — dashboards, alerts and anomaly detectors, SLOs, workflows, notebooks, API integrations, and classic auto-tagging / management zones.

   The AWS and Azure metric-and-entity mapping pages are the lookup for each rewrite; CLOUD-04 § 1 shows the Lambda keys side by side.
4. **Switch over** — disable the classic connection and *"Keep it disabled (not deleted) for one to two weeks as a rollback buffer."*
5. **Decommission** — delete the classic connection once nothing depends on it. Historical classic data stays queryable until it ages out under your retention settings.

Plan the overlap deliberately. Classic and new connections can run side by side across *different* accounts, but the guide advises against running both against the *same* account, subscription or project for long: it risks *"Duplicated cost"* and *"Cloud API throttling"*. Keep the overlap short. One gap remains on AWS: CloudWatch Metric Streams is not yet supported by new AWS connections, so an account that depends on it keeps that classic path for now (CLOUD-02 § 7).

To see which key scheme an environment is ingesting before you rewrite anything, group cloud metric keys by `dt.da.source`. Rows with a `dt.da.source` value come from new connections. Rows without one come from a classic path: `dt.cloud.*` keys from classic built-in services, or `cloud.*` keys from classic cloud services and classic Metric Streams.

```
metrics from:now()-9d
| filter startsWith(metric.key, "cloud.") or startsWith(metric.key,
"dt.cloud.")
| fieldsAdd scheme = if(startsWith(metric.key, "dt.cloud."), "dt.cloud.*
(classic)", else: "cloud.*")
| summarize keys = countDistinctExact(metric.key), by:{scheme, dt.da.source}
| sort scheme asc
```

Sources:

- Upgrade from classic to new cloud connections (DT docs) — *"Existing connections are not automatically upgraded or removed."*, *"Classic entity IDs and classic entity types do not carry over to new Smartscape nodes."*, *"Metric keys are unified to cloud.azure.. with a metric dimension dt.da.source == "azure-metric-poller" when using the new Azure connection."*, *"Keep it disabled (not deleted) for one to two weeks as a rollback buffer."*, *"Not yet supported by new AWS connections."*
- AWS metric and entity mapping (DT docs), Azure metric and entity mapping (DT docs)
- Key-scheme query executed on the validation tenant 11/11/2026: 93 `cloud.aws.*` keys with `aws-metric-poller`, 62 `cloud.azure.*` keys with `azure-metric-poller`, and, with no `dt.da.source`, 6 `dt.cloud.*` keys plus 2 lowercase classic cloud-service keys

> (`cloud.azure.microsoft_storage.storageaccounts.transactions`, `…usedcapacity`) over 9 days.

3. Cloud Entity Model

Dynatrace maps cloud resources to its entity model. Each cloud resource becomes a monitored entity with relationships to other entities.

AWS Entity Types

Entity Type	Description
`dt.entity.ec2_instance`	EC2 virtual machines
`dt.entity.aws_lambda_function`	Lambda functions
`dt.entity.relational_database_service`	RDS instances
`dt.entity.elastic_load_balancer`	ALB/NLB/CLB load balancers
`dt.entity.aws_availability_zone`	Availability zones
`dt.entity.custom_device`	Other AWS services via CloudWatch

Azure Entity Types

Entity Type	Description
`dt.entity.azure_vm`	Virtual machines
`dt.entity.azure_web_app`	App Service instances
`dt.entity.azure_sql_database`	SQL Database instances
`dt.entity.azure_cosmos_db`	Cosmos DB accounts
`dt.entity.azure_load_balancer`	Load balancers

GCP Entity Types

GCP is the one provider whose classic entity types do **not** follow the `dt.entity.<service>` shape the AWS
and Azure tables above use. Managed GCP services are modelled under a colon-delimited namespace —
`dt.entity.cloud:gcp:<service>` — which is not guessable from the AWS/Azure pattern:

Entity Type	Description
`dt.entity.cloud_application`	Kubernetes workloads (GKE and any other monitored cluster) — not Cloud Run, whose OneAgent-monitored instances appear as hosts
`dt.entity.cloud_application_instance`	Individual pods/instances
`dt.entity.cloud:gcp:gce_instance`	Compute Engine VMs
`dt.entity.cloud:gcp:cloud_function`	Cloud Functions
`dt.entity.cloud:gcp:cloud_run_revision`	Cloud Run revisions
`dt.entity.cloud:gcp:cloudsql_database`	Cloud SQL databases
`dt.entity.cloud:gcp:gcs_bucket`	Cloud Storage buckets
`dt.entity.cloud:gcp:pubsub_topic` / `…:pubsub_subscription`	Pub/Sub
`dt.entity.cloud:gcp:k8s_cluster` / `…:k8s_pod` / `…:k8s_node`	GKE topology
`dt.entity.gcp_zone`, `dt.entity.google_compute_engine`	Zone and GCE roll-ups

> ⚠️ `dt.entity.google_cloud_platform_service` **does not exist**, and a query against it does not error —
> it returns zero rows behind a *"The entity type … wasn't found"* warning, which reads as an empty
> environment. Corrected 08/27/2026. This is a partial list: 50+ GCP entity models exist (55 on the validation tenant, 09/24/2026). Enumerate the
> real set rather than guessing a name:

```
dql &gt; fetch dt.semantic_dictionary.models &gt; | filter startsWith(name, "dt.entity.") and contains(name, "gcp") &gt; | fields name &gt; | sort name asc &gt;
```

4. Cloud Metrics vs Host Metrics

Understanding the difference between cloud-sourced metrics and OneAgent host metrics is critical for accurate monitoring.

Aspect	Cloud Metrics	Host Metrics (OneAgent)
Source	Cloud provider API (CloudWatch, Azure Monitor, etc.)	OneAgent on the host
Granularity	1-5 minutes	10-60 seconds
Scope	Infrastructure + managed services	OS, processes, code-level
Metric namespace	`cloud.aws.*`, `cloud.azure.*`, `cloud.gcp.*`	`dt.host.*`, `dt.process.*`
Cost	Cloud API call costs	No additional cloud cost
Coverage	All resources (even without agent)	Only instrumented hosts

Best Practice: Use Both

For compute resources (EC2, Azure VMs, GCE), use **both** cloud integration and OneAgent:
- Cloud integration provides the **cloud context** (instance type, availability zone, tags)
- OneAgent provides **deep observability** (processes, traces, code-level diagnostics)
- Dynatrace automatically correlates both data sources via the entity model

> **Sources:** Built-in metrics on Grail (DT docs) — the `builtin:` → `dt.` naming transformation, which is what decides whether a metric key from a classic example still resolves, AWS metrics ingest (DT docs) — the polling ingestion path and its granularity, CloudWatch Metric Streams (DT docs) — the streaming path, which uses a **different key scheme** (see CLOUD-04 § 1). **Derived:** the use-both

> recommendation for compute resources combines the two coverage models; no page states it as a recommendation.

Let's query the cloud entities currently monitored in your environment.

5. Querying Cloud Entities

The following queries demonstrate how to explore cloud entities in your Dynatrace environment.

Count EC2 Instances

```
// Count all monitored EC2 instances
fetch dt.entity.ec2_instance, from:-7d
| summarize instance_count = count()

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.
// Smartscape note (dt.entity.* is deprecated but still functional): this
resource IS a Smartscape
// node — AWS_EC2_INSTANCE, AWS_RDS_DBINSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type, uppercased). On
Clouds-app connections the
// classic dt.entity.* type can under-count or return nothing for the same
estate (validation
// tenant 09/24/2026: RDS 0 classic vs 3 Smartscape; EC2 6 vs 25 running).
Prefer the Smartscape
// query.
```

Count Azure VMs

```
// Count all monitored Azure VMs
fetch dt.entity.azure_vm, from:-7d
| summarize vm_count = count()

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.
// Smartscape note (dt.entity.* is deprecated but still functional): this
resource IS a Smartscape
// node — AWS_EC2_INSTANCE, AWS_RDS_DBINSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type, uppercased). On
Clouds-app connections the
// classic dt.entity.* type can under-count or return nothing for the same
estate (validation
// tenant 09/24/2026: RDS 0 classic vs 3 Smartscape; EC2 6 vs 25 running).
Prefer the Smartscape
// query.
```

Compare Cloud Resources Across Providers

This query uses `append` to combine entity counts from multiple cloud providers into a single view.

```
// Compare cloud resource counts across providers
fetch dt.entity.ec2_instance, from:-7d
| summarize resource_count = count()
| fieldsAdd provider = "AWS", resource_type = "Compute (EC2)"
| append [
    fetch dt.entity.azure_vm, from:-7d
    | summarize resource_count = count()
    | fieldsAdd provider = "Azure", resource_type = "Compute (VM)"
  ]
| append [
    fetch dt.entity.aws_lambda_function, from:-7d
    | summarize resource_count = count()
    | fieldsAdd provider = "AWS", resource_type = "Serverless (Lambda)"
  ]
| sort provider asc

// Time range required (corrected 08/12/2026): dt.entity.* is an event-
LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without
an explicit from:
// this counted 5 of 13 EC2 instances against the notebook default window and
looked correct.

// Smartscape note (dt.entity.* is deprecated but still functional): these
resources ARE
// Smartscape nodes — AWS_EC2_INSTANCE,
AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES,
// AWS_LAMBDA_FUNCTION, AZURE_MICROSOFT_WEB_SITES (CloudFormation / ARM type,
uppercased). On
// Clouds-app connections the classic dt.entity.* types can under-count or
return nothing for the
// same estate (validation tenant 09/24/2026: EC2 6 classic vs 25 running on
Smartscape; Lambda 2
// vs 34; Azure VM 1 vs 8; Web App 0 vs 2). Prefer smartscapeNodes for the
cloud-resource inventory.
```

Host CPU Usage Across Cloud Hosts

This query retrieves average CPU usage for all monitored hosts, which includes both cloud and on-premises hosts.

```
// Average CPU usage across all hosts in the last hour
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

6. Multi-Cloud Strategy

When monitoring multiple cloud providers with Dynatrace, consider these best practices:

Naming Conventions

Establish consistent naming across providers:

Element	Convention	Example
Tags	`cloud-provider:aws`, `cloud-provider:azure`	Identify cloud origin
Segments	`Cloud - AWS - Production`, `Cloud - Azure - Staging`	Scope views by provider/env (filtering only — access is granted by IAM policies, not segments; Management Zones are the classic equivalent)
Workflow triggers	`cloud-aws-critical`, `cloud-azure-warning`	Route problems by provider with problem-triggered workflows (alerting profiles are the classic equivalent)

Deployment Pattern

Component	Recommended Deployment
Environment ActiveGate	Only for classic polling (Managed / strict network isolation): one per provider per region; not needed for Clouds-app connections on SaaS
OneAgent	On all compute instances (EC2, Azure VM, GCE)
Cloud Integration	Enabled per provider with least-privilege IAM
Log Forwarding	Centralized via OpenPipeline

Cost Considerations

- **Monitor only what you need** — disable unused service monitoring to reduce API calls
- **Use tag-based filtering** — monitor only tagged resources (e.g., `monitored:true`)
- **Review cloud API costs** — CloudWatch API calls, Azure Monitor queries, and GCP monitoring API calls all have costs
- **Right-size polling intervals** — not every metric needs 1-minute granularity

7. Summary and Next Steps

Key Takeaways

- Dynatrace integrates with cloud providers via the **Clouds app** (direct connections, recommended) or **classic ActiveGate polling** (legacy/Managed)
- Cloud resources are mapped to the **Dynatrace entity model** with automatic relationship discovery
- **Cloud metrics** complement **OneAgent host metrics** — use both for full visibility
- A consistent **naming convention** and **tagging strategy** is essential for multi-cloud governance

Next Steps

- **CLOUD-02: AWS Integration** — Deep dive into AWS-specific setup and monitoring
- **CLOUD-05: Azure Integration** — Azure Monitor integration and supported services
- **CLOUD-06: GCP Integration** — Google Cloud monitoring configuration

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.